

Estrutura do Servidor

.env - Variáveis de ambiente (Chaves de API, DB URL, etc.) - PROTEGIDO

.gitignore - Ignora ficheiros sensíveis como .env e node_modules

README.md - Documentação geral do projeto

TESTS.md - Plano de testes detalhado (ver item 3)

src/

app.js - Inicialização e configuração do servidor (Express, etc.)

server.js - Ponto de entrada (Bootstrap do servidor)

config/ - Configurações de conexões externas

database.js - Ligação ao Banco de Dados (PostgreSQL/MySQL)

routes/ - Configuração de Rotas (Endpoints da API)

driverRoutes.js

fleetRoutes.js

deliveryRoutes.js

controllers/ - Camada de Controlo (Valida requisições e envia respostas JSON)

DriverController.js

FleetController.js

DeliveryController.js

services/ - Camada de Negócio (Lógica de rotas e regras da LogiTech)

DriverService.js

FleetService.js

DeliveryService.js

repositories/ - Camada de Dados (Repository Pattern - isola o SQL)

- DriverRepository.js - [FleetRepository.js](#) - DeliveryRepository.js

CÓDIGO JAVAScript

```
const express = require('express');
const router = express.Router();
// const DeliveryController = require('../controllers/DeliveryController');

/**
 * @route POST /api/deliveries
 * @desc Cria uma nova recolha/entrega de alta prioridade no sistema.
 * @access Privado (Requer Token JWT válido)
 */
router.post('/', (req, res) => {
  // COMENTÁRIO: Recebe o payload JSON validado, envia para o DeliveryController
  // para processar a lógica de negócio através do Service.
  res.status(201).json({ message: "Rota de criação de entrega pronta para implementação."
});
});

/**
 * @route PUT /api/deliveries/:id/status
 * @desc Atualiza o status de uma entrega (Fase de Apagão de Comunicação resolvida aqui).
 * @access Privado (Aplicações dos Motoristas)
 */
router.put('/:id/status', (req, res) => {
  // COMENTÁRIO: Padroniza o payload recebido do motorista. Evita requisições bagunçadas.
  // Irá acionar o service correspondente para atualizar o banco de dados via Repository.
  res.status(200).json({ message: "Rota de atualização de status configurada." });
});

module.exports = router;
```

CÓDIGO MySQL

-- Criação da tabela de Motoristas (Garante unicidade e sem duplicados)

```
CREATE TABLE motoristas (  
    id_motorista SERIAL PRIMARY KEY,  
    nome VARCHAR(150) NOT NULL,  
    documento_identificacao VARCHAR(20) UNIQUE NOT NULL, -- Impede duplicidade  
    telefone VARCHAR(20),  
    data_cadastro TIMESTAMP DEFAULT CURRENT_TIMESTAMP  
);
```

-- Criação da tabela de Frotas

```
CREATE TABLE frotas (  
    id_frota SERIAL PRIMARY KEY,  
    placa_veiculo VARCHAR(10) UNIQUE NOT NULL,  
    modelo VARCHAR(50) NOT NULL  
);
```

-- Saneamento de Dados: Tabela para gerir ferramentas atomizadas (Fim das strings por vírgula)

```
CREATE TABLE itens_inventario (  
    id_item SERIAL PRIMARY KEY,  
    nome_ferramenta VARCHAR(100) UNIQUE NOT NULL
```

```
);
```

```
-- Tabela associativa para inventário de ferramentas por frota (1ª Forma Normal aplicada)
```

```
CREATE TABLE inventario_frota (
```

```
    id_frota INT REFERENCES frotas(id_frota) ON DELETE CASCADE,
```

```
    id_item INT REFERENCES itens_inventario(id_item) ON DELETE CASCADE,
```

```
    quantidade INT NOT NULL CHECK (quantidade >= 0),
```

```
    PRIMARY KEY (id_frota, id_item)
```

```
);
```

```
-- Tabela de Entregas (Controle de Rotas de Alta Prioridade)
```

```
CREATE TABLE entregas (
```

```
    id_entrega SERIAL PRIMARY KEY,
```

```
    id_motorista INT REFERENCES motoristas(id_motorista),
```

```
    id_frota INT REFERENCES frotas(id_frota),
```

```
    status_entrega VARCHAR(30) NOT NULL DEFAULT 'Pendente', -- Ex: 'Em Trânsito',  
    'Entregue'
```

```
    coordenadas_atualizacao TEXT,
```

```
    ultima_atualizacao TIMESTAMP DEFAULT CURRENT_TIMESTAMP ON UPDATE  
    CURRENT_TIMESTAMP
```

```
);
```

1. Cenário: Saneamento e Cadastro de Motoristas

- **Caso de Teste 01:** Impedir cadastro duplicado de Motoristas.
- **Entrada:** Payload com um documento (CPF/CNH) já existente no banco de dados.
- **Resultado Esperado:** O sistema deve interceptar a requisição, retornar HTTP Status `409 Conflict` e um JSON estruturado com a mensagem: `{"error": "Motorista já cadastrado com este documento."}`.
- **Caso de Teste 02:** Validação de Payload de Motorista.
- **Entrada:** JSON com campo `nome` em falta ou em branco.
- **Resultado Esperado:** Retornar HTTP Status `400 Bad Request` e a mensagem detalhando o campo obrigatório em falta.

2. Cenário: Atualizações de Status (Fim do Gargalo de Comunicação)

- **Caso de Teste 03:** Processamento de requisições em alta concorrência.
- **Ação:** Simular 1000 requisições simultâneas de motoristas atualizando o status da rota via endpoint `/api/deliveries/:id/status``.
- **Resultado Esperado:** O Back-End deve responder a 100% das requisições com sucesso (HTTP `200 OK`) sem que o servidor caia ou trave, mantendo o tempo de resposta abaixo de 200ms por requisição.
- **Caso de Teste 04:** Padronização de mensagens de erro.
- **Entrada:** Envio de um status inválido (ex: `status: "Voando"`).
- **Resultado Esperado:** Retornar HTTP `422 Unprocessable Entity` com erro amigável: `{"error": "Status informado inválido. Utilize: 'Pendente', 'Em Trânsito' ou 'Concluído'."}`.

3. Cenário: Blindagem de Segurança

- **Caso de Teste 05:** Proteção de Endpoints Sensíveis.
 - **Ação:** Tentar acessar à rota `/api/deliveries` sem enviar o Token JWT no cabeçalho (Header Authorization).
 - **Resultado Esperado:** O middleware de segurança deve bloquear o acesso imediatamente, retornando HTTP `401 Unauthorized`.
-
- **Caso de Teste 06:** Proteção contra Vazamento de Credenciais.
 - **Ação:** Verificação estática do código (Pipeline de CI/CD).
 - **Resultado Esperado:** Garantir que nenhuma string de conexão ou token esteja hardcoded no código fonte. O sistema deve falhar ao iniciar se o arquivo `.env` não estiver presente localmente.